

# Data Analyst Interview Questions

Please prepare answers to the following questions ahead of the interview.

In the interview you will present and talk through each answer. Show SQL where a query is asked; short explanations where reasoning is asked. Work through the questions in order.

If you used external resources (search, documentation, AI, or similar) while preparing, say so in your presentation: what you did not know, and how you went about finding it.

# 1. Two left joins

Read and compare queries A and B.

Explain why their result sets might differ.

```
-- A
SELECT c.customer_id, c.name, o.order_id
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= '2024-01-01';

-- B
SELECT c.customer_id, c.name, o.order_id
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
AND o.order_date >= '2024-01-01';
```

## 2a. Most recent order

Write a SQL query on `orders` that returns the most recent order for each customer.

`order_id` is an auto-increment / identity column (newer orders have a higher `order_id`).

All columns are not nullable.

Return `customer_id`, `order_id`, `order_date`, `amount`.

Example rows in `orders`:

| order_id | customer_id | order_date | amount |
|----------|-------------|------------|--------|
| 10       | 1           | 2024-01-05 | 20.00  |
| 11       | 1           | 2024-02-10 | 35.00  |
| 12       | 2           | 2024-01-20 | 15.00  |
| 13       | 1           | 2024-03-01 | 40.00  |
| 14       | 2           | 2024-06-01 | 50.00  |

## 2b. Second most recent order

Using the same `orders` table, write a SQL query that returns the second most recent order for each customer.

Return `customer_id`, `order_id`, `order_date`, `amount`.

Customers with fewer than two orders should not appear.

# 3a. Accounts per tier per day

We have a product with 3 subscription tiers: bronze, silver, gold. We store the history of each account in `account_tier_scd`, using validity ranges.

When an account is created or altered, a new row is added. Any old rows have their validity range closed, and the current row is left with `valid_to` = null.

We also have `dim_date`: one row per calendar date. Its range is wider than the dates in `account_tier_scd`.

`account_tier_scd`

| account_id | tier   | valid_from | valid_to   |
|------------|--------|------------|------------|
| 101        | bronze | 2024-01-01 | 2024-03-15 |
| 101        | silver | 2024-03-15 | 2024-06-01 |
| 101        | gold   | 2024-06-01 | null       |
| 102        | silver | 2024-02-01 | null       |
| 103        | bronze | 2024-01-01 | 2024-04-01 |
| 103        | gold   | 2024-04-01 | null       |

`dim_date`

| date       |
|------------|
| 2023-12-01 |
| 2023-12-02 |
| ...        |
| 2025-12-31 |

Write a SQL query that counts how many accounts were on each tier for each date in 2024.

Include days with a count of zero.

Return `date`, `tier`, `account_cnt`.

### 3b. Silver to gold

Using the same `account_tier_scd` table, write a SQL query that finds every account that moved from **silver** to **gold** in the past 30 days.

The sample rows above illustrate structure only; they are not required to fall inside the 30-day window.

Return `account_id`, `changed_at`.

`changed_at` is the date the gold row started (`valid_from` on the gold row).

## 4. Git

1. What does each command do?

- `git pull`
- `git checkout`
- `git add`
- `git commit`
- `git push`
- `git merge`
- `git revert`

2. Explain a scenario that would cause a merge conflict.

3. How would you go about resolving that merge conflict?

## 5. Dashboard request

A request comes in for a new dashboard.

What questions do you ask, and what information do you need, before you start working?